

**ADDIS ABABA SCIENCE AND TECHNOLOGY UNIVERSITY
(AASTU)**

COLLEGE OF ENGINEERING

DEPARTMENT OF SOFTWARE ENGINEERING



INVENTORY MANAGEMENT SYSTEM

COURSE TITLE: Database Systems

COURSE CODE: SWEG2108

INSTRUCTOR'S NAME: Mr. Yaynshet

SECTION: A

GROUP MEMBERS	ID. NUMBER
1. Blen Mekonnen	ETS 0357/17
2. Biniam Girma	ETS 0312/17
3. Abemelek Samson	ETS 0043/17
4. Adonias Hailu	ETS 0103/17
5. Befikir Shimelis	ETS 0237/17
6. Eden Wubshet	ETS 0461/17

SUBMITTED ON: APRIL 30, 2026

Table of Contents

ABSTRACT	4
INTRODUCTION	5
Background of the Organization	6
Problem Statement	6
OBJECTIVES	7
SCOPE and LIMITATION	7
Benefits Of The System	7
Methodology And Tools	8
Methodology	8
Tools	8
Entities and relationships	9
Our Entities	9
Relationships:	9
Logical schema	11
Normalization	12
First Normal Form (1NF)	12
Second Normal Form (2NF)	12
Third Normal Form (3NF)	13
Boyce-Codd Normal Form (BCNF)	13
Input Forms, Outputs, And Reports	14
Input Forms	14
Outputs	14
Reports	14
IMPLEMENTATION	15
Implementation in MySQL	15
Database and Table Creation	15
Data Insertion	16
Query execution	16
Implementation in MongoDB	17

Collections and data insertion	17
Query Execution and aggregation	18
Testing	19
Results	19
Challenges and improvements	19
References	20
Annex A – Analysis Questions	21
Annex B – Forms	21
Annex C – SQL Code	22
Annex D – MongoDB Code	26

ABSTRACT

This project focuses on the design and implementation of a database-driven Inventory Management System for an organization. The system aims to replace manual record-keeping with a structured and efficient database solution. It involves analyzing data requirements, designing an entity-relationship model, normalizing the database, and implementing it using MySQL and MongoDB. The system supports essential operations such as storing, retrieving, updating, and deleting inventory data, and demonstrates improved accuracy, organization, and efficiency.

INTRODUCTION

Starting off from the basics, Inventory simply means the goods, materials, or products that a business keeps so it can operate or sell. It refers to anything from raw materials in a factory to finished items on a shop shelf. **Inventory Management** is the process of tracking, organizing, and controlling those items so the business always knows what it has, what it needs, and when to restock. It is used almost everywhere including shops, warehouses, hospitals, schools, even small campus stores, so it is safe to say any place that deals with physical items has an inventory management system, manual or digitalized. Inventory management is very important because poor inventory handling leads to wasted money, lost sales, or running out of essential items at the worst time. Businesses use inventory systems to avoid overstocking, prevent shortages, reduce losses, and make better decisions. That's where a database system becomes useful: instead of relying on messy, scattered manual records, a database keeps everything organized, accurate, and easy to access in real time. Without a system like this, businesses run the risk of data loss, human errors, difficulty tracking stock levels, slow searching, and also poor reporting.

A well-designed database solves these with ease. By storing structured data, allowing fast retrieval, ensuring consistency, and supporting operations like searching, updating, and reporting efficiently. We chose an inventory management system for our database project because it is practical, widely used, and it clearly demonstrates key database concepts such as data organization, relationships, and real-time updates. It also solves real world problems in a simple but meaningful way. Our database main point is to turn raw data into something reliable, organized, and useful.

Background of the Organization

The organization we selected is the campus shop by the container that serves students and staff by providing number of goods such as snacks, drinks , cleaning supplies, and many other. The shop works in a small but busy environment where demand changes frequently. Currently, inventory records are managed manually using notebooks. This makes it difficult to accurately monitor the amount of stock they have, update item quantities in real time, and get information about it quickly when needed. As a result, challenges such as stock inconsistencies, delays in updating records, and sometimes shortages or overstocking are common. With the increasing number of transactions and growing demand, the shop requires a more structured and efficient system. Therefore, introducing a database-based inventory management system is essential to improve accuracy, speed, and overall organization of inventory operations.

Problem Statement

In our country Ethiopia, well over 70-80% of small business owners and retailers track their inventory using paper, notebooks or scattered spreadsheets. This comes with a lot of hidden problems that slow a business down. Major issue is human error; numbers get incorrectly written, items are missed when making calculations or updates are forgotten. But even after that, searching for information is slow and frustrating, especially as the amount of stock grows.

From a real world perspective, lets take the shop at our campus, the owners/workers write down every sale and restock in a notebook. At first they thought it was manageable, but now that they have expanded, they have started to struggle keeping records updated. They spend extra time counting items manually and still end up with mismatch between whats written on their notebook and what's actually on the shelf. As the business gets bigger, manual inventory systems have proved to be no efficient or effective. This is exactly why businesses shift toward database driven solutions.

OBJECTIVES

The system addresses several common inventory challenges by keeping records accurate and easy to manage.

This project's main objective is to design and develop an efficient and reliable database-driven inventory management system that replaces manual record-keeping, improves accuracy, ensures easy data access, and supports better decision-making within a business environment.

- ✓ To store inventory data in a structured and organized manner.
- ✓ To reduce human errors associated with manual record-keeping.
- ✓ To enable fast searching and retrieval of product information.
- ✓ To simplify adding, updating, and deleting inventory records.
- ✓ To prevent data loss.
- ✓ To enhance overall efficiency and reduce time wasted on manual processes.

SCOPE and LIMITATION

This project covers the design and basic implementation of database system for inventory management system for businesses, mainly shops. It focuses on analyzing real world data requirements and modeling the system using ERD, and transforms the design to a database schemas. It also includes the creation of relation tables in MySQL and document based collections in MongoDB, along with sample data insertion and query execution to show core operations such as data storage, retrieval, updating, and deletion. However, our project does not include the development of a full user interface, or a complete software application. It is basically a backend database design and query based interaction. Multi user access, security, and such features are beyond our scope.

Benefits Of The System

The proposed system provides several advantages over the current manual method. It improves accuracy by reducing human errors in record-keeping and calculations. It saves time by allowing quick searching and updating of inventory data. It also enhances organization by storing all information in a structured database, making it easier to manage large amounts of stock. In addition, the system reduces the risk of data loss compared to paper-based records. Overall, it increases efficiency, supports better decision-making, and helps the shop operate more smoothly as it grows.

Methodology And Tools

Methodology

Our project follows a structured database development approach to design and implement an inventory management system. The process begins with requirement analysis, where the needs of a typical campus shop are identified, including tracking stock levels, updating inventory, and retrieving product information efficiently. After understanding the requirements, system design is carried out using an Entity Relationship Diagram (ERD) to define entities, attributes, and relationships. The ERD is then transformed into a logical database schema to ensure proper organization of data.

Once the design is finalized, implementation is done using both relational and non-relational database approaches. In the relational model, tables are created in MySQL, while in the non-relational model, collections are created in MongoDB. Sample data is then inserted to simulate real world experience, and SQL and NoSQL queries are executed to test core operations such as insertion, updating, deletion, and retrieval of data. Finally, testing is performed to ensure that the system behaves as expected and supports efficient inventory management operations.

Tools We Used

- **MySQL** – Used for designing and implementing the relational database, including tables and SQL queries
- **MongoDB** – Used for implementing a document-based database structure to store and manage inventory data
- **ERD Tools**– Used to design the Entity Relationship Diagram
- **VS Code** – Used for writing and testing queries and related implementation code
- **Windows OS / Local Environment** – Used as the development environment for building and testing the system

Entities and relationships

Our inventory management system for the campus shop is designed based on key entities representing the objects involved in the shop operations we see in the real world. As we know, entities are the core building block of the **Entity Relationship Model**. They help us organize and structure the data in a meaningful way.

Our Entities are:

- **Product:** The items sold in the shop.
 - Its attributes are
 - Product identifier
 - Name of the product
 - Price of the product
- **Supplier:** The sources from which the product are purchased.
 - Its attributes are
 - Supplier identifier
 - Name of the supplier
 - Phone number of the supplier
- **Inventory:** This keeps track of the stock levels of each product.
 - Its attributes are
 - Inventory identifier
 - Quantity
- **Sale:** Records each transaction made in the shop.
 - Its attributes are
 - Sale identifier
 - Quantity sold
 - Date of sale
- **User:** Customer
 - Its attributes are
 - Customer identifier
 - Name of the customer
 - Phone number of the customer

Relationships:

- A Supplier can supply many products (1:M).
- A Product is linked to an inventory to be kept track off. Assuming there are no multiple storage locations or different stock batches (1:1).

- A Product can appear in many sales records (1:M).
 - A User (customer) can make many sales (1:M).
- ER (Entity Relationship) Diagram

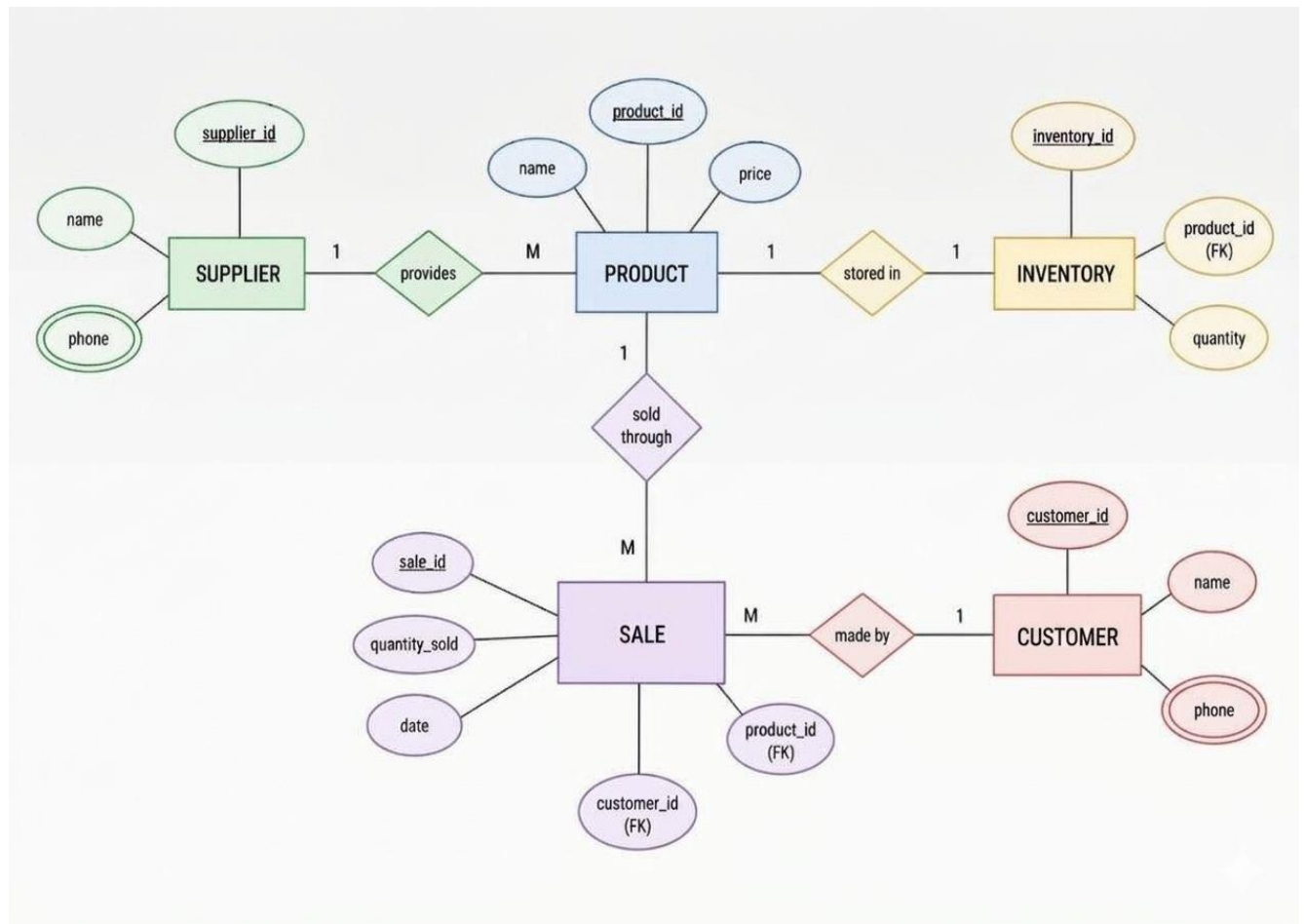


Diagram 1: ER diagram of the inventory system

Logical schema

A logical data model or logical schema is a data model of a specific problem domain expressed independently of a particular database management product or storage technology (physical data model) but in terms of data structures such as relational tables and columns, object-oriented classes, or XML tags. This is as opposed to a conceptual data model, which describes the semantics of an organization without reference to technology.

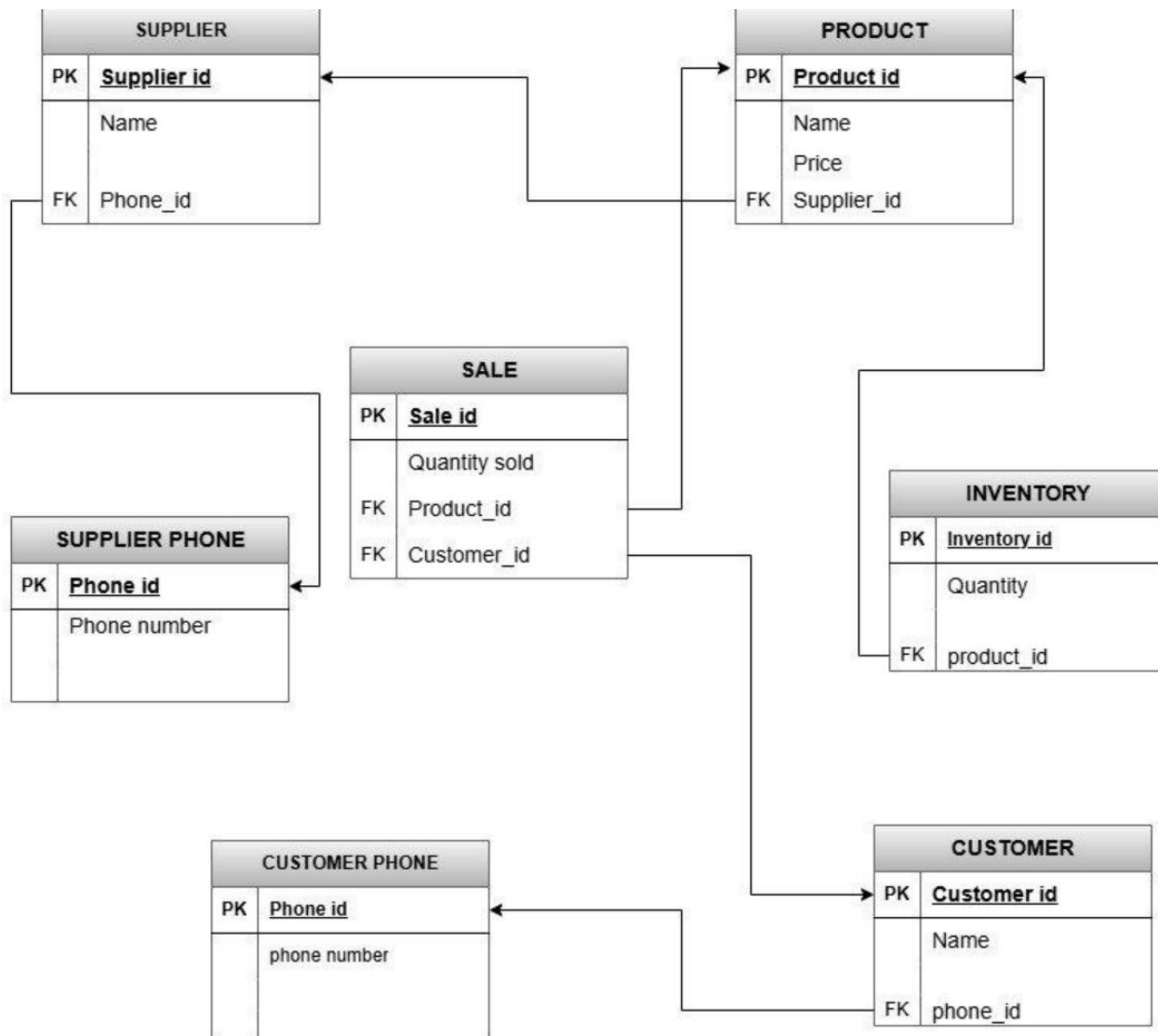


Diagram 2: Logical schema

Normalization

Normalization is the process of organizing data in a database to reduce redundancy and improve data integrity. It ensures that the database structure is efficient and free from anomalies such as insertion, deletion, and update problems.

Database normalization is structured around a series of increasingly strict rules called normal forms. Each normal form addresses specific types of redundancy and dependency issues, guiding you toward a more robust and maintainable relational schema. The most widely applied normal forms are First Normal Form (1NF), Second Normal Form (2NF), Third Normal Form (3NF), and Boyce-Codd Normal Form (BCNF).

First Normal Form (1NF)

This is the initial stage in the process of database normalization. It ensures that each column in a table contains only atomic, indivisible value, and that each row is uniquely identifiable. It lays the groundwork for a more organized and consistent database structure.

Key Requirements:

- All columns contain atomic values (no lists, sets, or composite fields).
- Each row is unique (typically enforced by a primary key).
- No repeating groups or arrays within a row.
- Each column contains values of a single data type.

In this system, all tables (Product, Supplier, Inventory, Sale, and User) satisfy 1NF because each field contains a single value, and there are no repeating or grouped attributes. Multi-valued attributes (phone) are separated into SUPPLIER_PHONE and CUSTOMER_PHONE. So that the schema is in 1NF.

Second Normal Form (2NF)

A table is in 2NF if it is in 1NF and every non-prime attribute (i.e., non-primary key attribute) is fully functionally dependent on the entire primary key. This addresses *partial dependencies*, where a non-key attribute depends only on part of a composite key.

All tables in the system use a single primary key (e.g., product_id, sale_id), so there are no partial dependencies. Every attribute depends entirely on its respective primary key. Therefore, all tables satisfy 2NF.

Third Normal Form (3NF)

A table is in 3NF if it is in 2NF and all the attributes are functionally dependent only on the primary key, there are no transitive dependencies (i.e., non-key attributes depending on other non-key attributes).

Benefits:

- Removes transitive dependencies.
- Reduces data duplication.
- Improves data integrity and maintainability.

In this design, attributes such as supplier details are stored in the Supplier table and not repeated in the Product table. Similarly, user information is separated from the Sale table. This eliminates transitive dependencies, so the database satisfies 3NF.

Boyce-Codd Normal Form (BCNF)

BCNF is a stricter version of 3NF. A table is in BCNF if, for every non-trivial functional dependency $X \rightarrow Y$, X is a superkey. In other words, every determinant must be a candidate key. It addresses certain edge cases where 3NF does not eliminate all redundancy, particularly when there are overlapping candidate keys or complex dependencies.

In our system, all determinants (such as product_id, supplier_id, sale_id) are primary keys. There are no unusual dependencies where non-key attributes determine other attributes. Therefore, all tables satisfy BCNF.

Final BCNF Schema

SUPPLIER(supplier_id PK, name, phone_id FK)

SUPPLIER_PHONE(phone_id, phone number)

PRODUCT(product_id PK, name, price, supplier_id FK)

INVENTORY(product_id PK, quantity)

CUSTOMER(customer_id PK, name)

CUSTOMER_PHONE(phone_id, phone number)

SALE(sale_id PK, product_id FK, customer_id FK, quantity_sold, date)

Input Forms, Outputs, And Reports

The system is designed in a way that makes it easy to enter, update, and retrieve inventory data. Instead of using notebooks and manual records, the database allows a more organized and structured interaction through basic input forms and simple outputs.

Input Forms

Input forms are used to add and manage data in the system. They act as a simple way for the user to interact with the database without needing to understand technical details. For this project, the main inputs include product details, supplier information, stock quantities, and sales records. For example, when a new product arrives, its details can be added through the product form, and its quantity can be updated through the inventory form. Similarly, sales transactions are recorded using a sale form by entering the product and quantity sold. These forms help make sure that data is entered consistently and reduces mistakes compared to manual recording.

Outputs

Outputs refer to the information that the system displays after retrieving data from the database. The system can show a list of available products, their prices, and current stock levels. It can also display supplier details and sales records when needed. This makes it easier to quickly access information without going through notebooks or manual files.

Reports

Reports are used to summarize important data from the system. The system can generate simple reports such as current stock status, sales records over a certain period, and items that are running low. These reports help the shop understand what is selling, what needs to be restocked, and how inventory is changing over time.

- **Stock Report** – displays all products along with their current quantities available in the shop
- **Sales Report** – shows records of all sales transactions, including product, quantity sold, and date
- **Low Stock Report** – identifies products with low quantity levels that need restocking.
- **Product Report** – provides a list of all products with their details such as name and price.

- **Supplier Report** – shows supplier information and the products they supply.

IMPLEMENTATION

The system is implemented using both MySQL and MongoDB to show how the same data can be handled in relational and non-relational databases. The implementation includes creating the database, defining tables and collections, inserting sample data, running queries, and testing the system to make sure it works as expected.

Implementation in MySQL

Database and Table Creation

The database was created in MySQL based on the logical schema designed earlier. The main tables created are supplier, product, inventory, and sale. Each table includes the necessary attributes along with primary keys and foreign keys to maintain relationships between the data.

MySQL:

```
CREATE TABLE supplier (  
    supplier_id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(120) NOT NULL,  
    phone VARCHAR(20) NOT NULL UNIQUE  
);  
  
CREATE TABLE product (  
    product_id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(120) NOT NULL,  
    price DECIMAL(10,2) NOT NULL,  
    supplier_id INT NOT NULL,  
    FOREIGN KEY (supplier_id) REFERENCES supplier(supplier_id)  
);  
  
CREATE TABLE inventory (  
    inventory_id INT AUTO_INCREMENT PRIMARY KEY,  
    product_id INT NOT NULL UNIQUE,  
    quantity INT NOT NULL,  
    FOREIGN KEY (product_id) REFERENCES product(product_id)  
);
```

```
CREATE TABLE sale (  
    sale_id INT AUTO_INCREMENT PRIMARY KEY,  
    product_id INT NOT NULL,  
    quantity_sold INT NOT NULL,  
    sale_date DATE NOT NULL,  
    FOREIGN KEY (product_id) REFERENCES product(product_id)  
);
```

Data Insertion

Lets insert sample data.

MySQL:

```
INSERT INTO supplier (name, phone) VALUES  
('Alem Suppliers', '0911001001'),  
('Abel Distribution', '0911001002');
```

```
INSERT INTO product (name, price, supplier_id) VALUES  
('Rice 1kg', 120.001),  
('Cooking Oil 1L', 210.00, 1),  
('Laundry Soap', 65.00, 2);
```

```
INSERT INTO inventory (product_id, quantity) VALUES  
(1, 100),  
(2, 60),  
(3, 150);
```

```
INSERT INTO sale (product_id, quantity_sold, sale_date) VALUES  
(1, 3, '2026-04-22'),  
(2, 2, '2026-04-22'),  
(1, 5, '2026-04-23'),  
(3, 10, '2026-04-23');
```

Query execution

Different queries were excuted to check if the system works correctly.

1. Products with supplier:-

```
SQL:
SELECT p.product_id, p.name, s.name AS supplier_name
FROM product p
JOIN supplier s ON s.supplier_id = p.supplier_id;
```

2. Stock levels:-

```
SQL:
SELECT p.name, i.quantity
FROM product p
JOIN inventory i ON i.product_id = p.product_id;
```

3. Sales records:-

```
SQL:
SELECT p.name, s.quantity_sold, s.sale_date
FROM sale s
JOIN product p ON p.product_id = s.product_id
```

4. Low stock:-

```
SQL:
SELECT p.name, i.quantity
FROM product p
JOIN inventory i ON i.product_id = p.product_id
```

Implementation in MongoDB

Collections and data insertion

In MongoDB, the system was implemented using collections instead of tables. The main collections created are the supplier, product, inventory, and sale. Data was inserted in the form of documents, each representing a record.

```
JavaScript:
db.supplier.insertMany([
  {supplier_id: 1, name: "Alem Suppliers", phone: "0911001001"},
  {supplier_id: 1, name: "Abel Distribution", phone: "0911001002"}
]);
```

```
dp.product.insertMany([
  {product_id: 1, name: "Rice 1kg", price: 120, supplier_id: 1},
  { product_id: 2, name: "Cooking Oil 1L", price: 210, supplier_id: 1},
  { product_id: 3, name: "Laundry Soap", price: 65, supplier_id: 2}
]);
```

Query Execution and aggregation

To test the system, we performed both basic queries and aggregation operations.

```
JavaScript:
Db.product.aggregate([
  { $lookup: {
    from: "supplier",
    localField: "supplier_id",
    foreignField: "supplier_id",
    as: "supplier"
  }
},
  { $unwind: "$supplier"},
  {
    $project: {
      product_id: 1,
      name: 1,
      price: 1,
      supplier_name: "$supplier.name"
    }
  }
]);
```

Testing

After implementing the system in both MySQL and MongoDB, testing was carried out to ensure that all of the operations work correctly. Various actions such as inserting new records, updating existing data, deleting entries, and retrieving information were tested. The queries were executed using sample data to check whether the system returns accurate and consistent results.

The system was able to correctly display product details, stock levels and sales records. Both relational and non relational implementations produced expected outputs. This confirms that the system functions properly and meets the needed requirements.

Results

The implementation of the database system shows obvious improvement compared to the manual method that was being used previously. Data is stored in a structured and organized way, which makes it easier to manage and access. Retrieving information is faster and the chances of errors are reduced significantly since the system handles relationship and constraints automatically. The system also allows better tracking of stock levels and sales activities. This helps in identifying low stock items and understanding sales patterns. It is safe to say, our system provides a more efficient and reliable way of managing inventory in the campus shop.

Challenges and improvements

During the implementation process, we faced some challenges. Writing the correct SQL queries and managing foreign key relationships required a lot of attention. There were also issues related to inserting data in the correct order for the purpose of maintaining consistency between tables. In MongoDB, understanding how to write the coded proved to be difficult since we have not used nor learned it before. Aggregation operations such as \$lookup and \$group required special attention and practice. However, these challenges were resolved through testing, debugging and making necessary adjustment to the queries and data structure.

Our system works as expected but there are several improvements that can be made. One could be a graphical user interface. A graphical user interface can be added to make the system easier to use, especially for users without technical knowledge. Security features are also part of the backlog. The features could include user authentication and access control. In addition the system can be expanded to support multiple users and handle larger amounts of data. Features such as advanced reporting and real time updates can also be implemented to make the system more suitable for real world applications

References

- [1] I. Sommerville, *Software Engineering*, 10th ed. Boston, MA, USA: Pearson, 2016.
- [2] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Boston, MA, USA: Pearson, 2016.
- [3] T. Connolly and C. Begg, *Database Systems: A Practical Approach to Design, Implementation, and Management*, 6th ed. Harlow, UK: Pearson, 2015.
- [4] Oracle Corporation, “MySQL Documentation,” [Online]. Available: <https://dev.mysql.com/doc/>
- [5] MongoDB Inc., “MongoDB Documentation,” [Online]. Available: <https://www.mongodb.com/docs/>
- [6] M. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. New York, NY, USA: McGraw-Hill, 2020.
- [7] GeeksforGeeks, “Introduction of Database Normalization,” [Online]. Available: <https://www.geeksforgeeks.org/dbms/introduction-of-database-normalization/>
- [8] DigitalOcean, “Database Normalization Explained,” [Online]. Available: <https://www.digitalocean.com/community/tutorials/database-normalization>
- [9] W3Schools, “DBMS Data Schemas,” [Online]. Available: <https://www.w3schools.in/dbms/data-schemas>

Annex A – Analysis Questions

1. What is your role?
2. How often do you interact with inventory or sales systems?
3. How do you currently manage inventory?
4. What challenges do you face?
5. How easy is the system to use?
6. How clear is the system interface?
7. How quickly can you perform tasks?
8. Has the system improved inventory tracking?
9. Has the system reduced errors in stock management?
10. How reliable is the system?
11. Which features do you find most useful?
12. Are the low-stock alerts helpful?
13. How useful are the reports?
14. Overall, how satisfied are you with the system?
15. Would you recommend this system to others?
16. What improvements would you suggest?
17. What problems did you encounter?
18. Any additional comments?

Annex B – Forms

Product form

Field Name	Input Type
Product ID	Number
Product Name	Text
Price	Decimal
Supplier ID	Number

Supplier Form

Field Name	Input Type
Supplier ID	Number
Supplier Name	Text
Phone Number	Text

Inventory Form

Field Name	Input Type
Inventory ID	Number
Product ID	Number
Quantity	Number

Sale Form

Field Name	Input Type
Sale ID	Number
Product ID	Number
Quantity sold	Number
Sale Date	Date

Customer Form

Field Name	Input Type
Customer ID	Number
Customer Name	Text
Phone Number	Text

Annex C – SQL Code

C.1: Schema.sql

DROP TABLE IF EXISTS sale;

DROP TABLE IF EXISTS inventory;

DROP TABLE IF EXISTS product;

DROP TABLE IF EXISTS customer;

DROP TABLE IF EXISTS supplier;

CREATE TABLE supplier (
 supplier_id INT AUTO_INCREMENT
PRIMARY KEY,

 name VARCHAR(100) NOT NULL,
 phone VARCHAR(20) NOT NULL
UNIQUE
);
CREATE TABLE product (
 product_id INT AUTO_INCREMENT
PRIMARY KEY,
 supplier_id INT NOT NULL,
 name VARCHAR(100) NOT NULL,

```

        price DECIMAL(10,2) NOT NULL,
        CONSTRAINT product_supplier_fk
            FOREIGN KEY (supplier_id)
            REFERENCES
supplier(supplier_id)
            ON UPDATE CASCADE
            ON DELETE RESTRICT,
        CONSTRAINT product_price_check
CHECK (price >= 0)
);
CREATE TABLE customer (
        customer_id INT AUTO_INCREMENT
PRIMARY KEY,
        name VARCHAR(100) NOT NULL,
        phone VARCHAR(20) NOT NULL
UNIQUE
);
CREATE TABLE inventory (
        inventory_id INT AUTO_INCREMENT
PRIMARY KEY,
        product_id INT NOT NULL UNIQUE,
        quantity INT NOT NULL,
        CONSTRAINT inventory_product_fk
            FOREIGN KEY (product_id)
            REFERENCES
product(product_id)
            ON UPDATE CASCADE
            ON DELETE CASCADE,
        CONSTRAINT inventory_quantity_check
CHECK (quantity >= 0)
);
CREATE TABLE sale (
        sale_id INT AUTO_INCREMENT
PRIMARY KEY,

```

```

        product_id INT NOT NULL,
        customer_id INT NOT NULL,
        quantity_sold INT NOT NULL,
        sale_date DATETIME NOT NULL
DEFAULT CURRENT_TIMESTAMP,
        CONSTRAINT sale_product_fk
            FOREIGN KEY (product_id)
            REFERENCES
product(product_id)
            ON UPDATE CASCADE
            ON DELETE RESTRICT,
        CONSTRAINT sale_customer_fk
            FOREIGN KEY (customer_id)
            REFERENCES
customer(customer_id)
            ON UPDATE CASCADE
            ON DELETE RESTRICT,
        CONSTRAINT sale_quantity_check
CHECK (quantity_sold > 0)
);

```

C.2: Queries.sql

```

USE inventory_db;

-- sample data used for testing

INSERT INTO supplier (name, phone) VALUES
('Abebe Supplier', '0911123456'),
('Kebede Trading', '0922234567'),
('Hana Wholesale', '0933345678');

INSERT INTO customer (name, phone) VALUES
('Martha Bekele', '0911456789'),
('Samuel Tesfaye', '0922567890'),

```

```
('Lidya Alemu', '0933678901'),  
('Henok Abate', '0944789012');
```

```
INSERT INTO product (supplier_id, name, price)  
VALUES  
(1, 'Soap', 150.00),  
(2, 'Shampoo', 800.00),  
(1, 'Toothpaste', 100.00),  
(3, 'Sugar 1kg', 90.00),  
(2, 'Cooking Oil 1L', 500.00),  
(3, 'Biscuit Pack', 40.00);
```

```
INSERT INTO inventory (product_id, quantity)  
VALUES  
(1, 120),  
(2, 60),  
(3, 75),  
(4, 50),  
(5, 40),  
(6, 90);
```

```
INSERT INTO sale (product_id, customer_id,  
quantity_sold, sale_date) VALUES  
(1, 1, 5, '2026-04-20 09:30:00'),  
(2, 2, 3, '2026-04-20 13:10:00'),  
(3, 3, 4, '2026-04-21 10:25:00'),  
(4, 4, 2, '2026-04-21 16:40:00'),  
(5, 1, 6, '2026-04-22 11:05:00'),  
(6, 2, 7, '2026-04-22 15:55:00'),  
(1, 3, 8, '2026-04-23 13:20:00'),  
(3, 4, 6, '2026-04-23 17:45:00'),  
(2, 1, 5, '2026-04-24 10:10:00');
```

```
(5, 2, 4, '2026-04-24 12:30:00');
```

-- day to day operations

-- 1) add a new product from an existing supplier

```
INSERT INTO product (supplier_id, name, price)  
VALUES (1, 'Detergent Powder', 180.00);
```

-- 2) create initial stock for the product i just added

-- using max(product_id) is okay here because this is
a simple demo flow

```
INSERT INTO inventory (product_id, quantity)  
SELECT MAX(product_id), 35  
FROM product;
```

-- 3) restock an item

```
UPDATE inventory  
SET quantity = quantity + 20  
WHERE product_id = 2;
```

-- 4) record one sale

```
INSERT INTO sale (product_id, customer_id,  
quantity_sold, sale_date)  
VALUES (2, 3, 5, NOW());
```

-- 5) reduce stock after the sale

```
UPDATE inventory  
SET quantity = quantity - 5  
WHERE product_id = 2;
```

-- 6) check current stock with supplier info

```

SELECT
    p.product_id,
    p.name AS product,
    s.name AS supplier,
    p.price,
    i.quantity AS stock_left
FROM product p
JOIN supplier s ON s.supplier_id = p.supplier_id
JOIN inventory i ON i.product_id = p.product_id
ORDER BY p.product_id;

```

-- simple analysis queries

-- 7) low stock check (threshold under 30)

```

SELECT
    p.product_id,
    p.name AS product,
    i.quantity AS stock_left
FROM product p
JOIN inventory i ON i.product_id = p.product_id
WHERE i.quantity < 30
ORDER BY i.quantity ASC;

```

-- 8) revenue summary by product

```

SELECT
    p.product_id,
    p.name AS product,
    SUM(sa.quantity_sold) AS units_sold,
    ROUND(SUM(sa.quantity_sold * p.price),
2) AS revenue
FROM sale sa

```

```

JOIN product p ON p.product_id = sa.product_id
GROUP BY p.product_id, p.name
ORDER BY revenue DESC;

```

-- 9) daily sales trend (units + revenue)

```

SELECT
    DATE(sa.sale_date) AS day,
    SUM(sa.quantity_sold) AS units_sold,
    ROUND(SUM(sa.quantity_sold * p.price),
2) AS revenue
FROM sale sa
JOIN product p ON p.product_id = sa.product_id
GROUP BY DATE(sa.sale_date)
ORDER BY day;

```

-- 10) stock value per product and grand total

```

SELECT
    p.product_id,
    p.name AS product,
    i.quantity,
    p.price,
    ROUND(i.quantity * p.price, 2) AS
stock_value
FROM product p
JOIN inventory i ON i.product_id = p.product_id
ORDER BY stock_value DESC;

SELECT ROUND(SUM(i.quantity * p.price), 2) AS
inventory_total_value
FROM product p
JOIN inventory i ON i.product_id = p.product_id;

```

```
-- 11) supplier with the highest sales revenue

SELECT
    s.supplier_id,
    s.name AS supplier,
    ROUND(SUM(sa.quantity_sold * p.price),
2) AS revenue
FROM supplier s
JOIN product p ON p.supplier_id = s.supplier_id
JOIN sale sa ON sa.product_id = p.product_id
GROUP BY s.supplier_id, s.name
ORDER BY revenue DESC;
```

-- required report queries from the checklist

-- 12) products with their suppliers

```
SELECT
    p.name AS product,
    s.name AS supplier
FROM product p
JOIN supplier s ON p.supplier_id = s.supplier_id
```

```
ORDER BY p.name;
```

-- 13) current inventory per product

```
SELECT
    p.name AS product,
    i.quantity
FROM inventory i
JOIN product p ON i.product_id = p.product_id
ORDER BY p.name;
```

-- 14) sales report with product, quantity, and date

```
SELECT
    p.name AS product,
    c.name AS customer,
    sa.quantity_sold AS sold_quantity,
    DATE(sa.sale_date) AS date
FROM sale sa
JOIN product p ON sa.product_id = p.product_id
JOIN customer c ON sa.customer_id = c.customer_id
ORDER BY sa.sale_date;
```

Annex D – MongoDB Code

D.1: Queries.js

// Inventory and Sales MongoDB Setup + Queries

// Run in mongosh: use inventory_db;
load("mongodb/queries.js")

```
db.supplier.drop();
db.customer.drop();
db.product.drop();
db.inventory.drop();
```

```
db.sale.drop();
```

```
db.supplier.insertMany([
    { supplier_id: 1, name: "Alem Suppliers", phone:
"0911001001" },
    { supplier_id: 2, name: "Abel Distribution", phone:
"0911001002" }
]);
```

```
db.customer.insertMany([
```

```

    { customer_id: 1, name: "Marta Bekele", phone:
"0912002001" },

    { customer_id: 2, name: "Nahom Demeke", phone:
"0912002002" },

    { customer_id: 3, name: "Saron Tola", phone:
"0912002003" }

]);

```

```

db.product.insertMany([

    { product_id: 1, name: "Rice 1kg", price: 120,
supplier_id: 1 },

    { product_id: 2, name: "Cooking Oil 1L", price:
210, supplier_id: 1 },

    { product_id: 3, name: "Laundry Soap", price: 65,
supplier_id: 2 }

]);

```

```

db.inventory.insertMany([

    { inventory_id: 1, product_id: 1, quantity: 100 },

    { inventory_id: 2, product_id: 2, quantity: 60 },

    { inventory_id: 3, product_id: 3, quantity: 150 }

]);

```

```

db.sale.insertMany([

    { sale_id: 1, product_id: 1, customer_id: 1,
quantity_sold: 3, date: "2026-04-22" },

    { sale_id: 2, product_id: 2, customer_id: 2,
quantity_sold: 2, date: "2026-04-22" },

    { sale_id: 3, product_id: 1, customer_id: 3,
quantity_sold: 5, date: "2026-04-23" },

    { sale_id: 4, product_id: 3, customer_id: 1,
quantity_sold: 10, date: "2026-04-23" }

]);

```

```

print("--- Products with Supplier ---");

printjson(

db.product.aggregate([

    {

        $lookup: {

            from: "supplier",

            localField: "supplier_id",

            foreignField: "supplier_id",

            as: "supplier"

        }

    },

    { $unwind: "$supplier" },

    {

        $project: {

            _id: 0,

            product_id: 1,

            name: 1,

            price: 1,

            supplier_name: "$supplier.name"

        }

    }

]).toArray()

);

```

```

print("--- Inventory by Product ---");

printjson(

db.inventory.aggregate([

    {

        $lookup: {

            from: "product",

```

```

        localField: "product_id",
        foreignField: "product_id",
        as: "product"
    }
},
{ $unwind: "$product" },
{
    $project: {
        _id: 0,
        inventory_id: 1,
        product_id: 1,
        product_name: "$product.name",
        quantity: 1
    }
}
]).toArray()
);

print("--- Sales with Product and Customer ---");
printjson(
    db.sale.aggregate([
        {
            $lookup: {
                from: "product",
                localField: "product_id",
                foreignField: "product_id",
                as: "product"
            }
        },
        {
            $lookup: {
                from: "customer",
                localField: "customer_id",
                foreignField: "customer_id",
                as: "customer"
            }
        },
        { $unwind: "$product" },
        { $unwind: "$customer" },
        {
            $project: {
                _id: 0,
                sale_id: 1,
                product_name: "$product.name",
                customer_name: "$customer.name",
                quantity_sold: 1,
                date: 1
            }
        },
        { $sort: { sale_id: 1 } }
    ]).toArray()
);

print("--- Total Sold By Product ---");
printjson(
    db.sale.aggregate([
        {
            $group: {
                _id: "$product_id",
                total_sold: { $sum: "$quantity_sold" }
            }
        }
    ])
);

```

```

    }
  },
  {
    $lookup: {
      from: "product",
      localField: "_id",
      foreignField: "product_id",
      as: "product"
    }
  },
  { $unwind: "$product" },
  {
    $project: {
      _id: 0,
      product_id: "$product.product_id",
      product_name: "$product.name",
      total_sold: 1
    }
  },
  { $sort: { total_sold: -1 } }
]).toArray()
);

```

```

print("--- Sales Count By Customer ---");
printjson(
  db.sale.aggregate([
    {
      $group: {
        _id: "$customer_id",
        sales_count: { $sum: 1 }
      }
    }
  ])
);

```

```

    }
  },
  {
    $lookup: {
      from: "customer",
      localField: "_id",
      foreignField: "customer_id",
      as: "customer"
    }
  },
  { $unwind: "$customer" },
  {
    $project: {
      _id: 0,
      customer_id: "$customer.customer_id",
      customer_name: "$customer.name",
      sales_count: 1
    }
  },
  { $sort: { sales_count: -1, customer_id: 1 } }
]).toArray()
);

```

D.2:Collections.json

```

{
  "supplier": [
    {
      "supplier_id": 1,
      "name": "Alem Suppliers",
      "phone": "0911001001"
    },
  ],
}

```

```

{
  "supplier_id": 2,
  "name": "Abel Distribution",
  "phone": "0911001002"
}
],
"customer": [
{
  "customer_id": 1,
  "name": "Marta Bekele",
  "phone": "0912002001"
},
{
  "customer_id": 2,
  "name": "Nahom Demeke",
  "phone": "0912002002"
},
{
  "customer_id": 3,
  "name": "Saron Tola",
  "phone": "0912002003"
}
],
"product": [
{
  "product_id": 1,
  "name": "Rice 1kg",
  "price": 120,
  "supplier_id": 1
},

```

```

{
  "product_id": 2,
  "name": "Cooking Oil 1L",
  "price": 210,
  "supplier_id": 1
},
{
  "product_id": 3,
  "name": "Laundry Soap",
  "price": 65,
  "supplier_id": 2
}
],
"inventory": [
{
  "inventory_id": 1,
  "product_id": 1,
  "quantity": 100
},
{
  "inventory_id": 2,
  "product_id": 2,
  "quantity": 60
},
{
  "inventory_id": 3,
  "product_id": 3,
  "quantity": 150
}
],

```

```
"sale": [
  {
    "sale_id": 1,
    "product_id": 1,
    "customer_id": 1,
    "quantity_sold": 3,
    "date": "2026-04-22"
  },
  {
    "sale_id": 2,
    "product_id": 2,
    "customer_id": 2,
    "quantity_sold": 2,
    "date": "2026-04-22"
  },
  {
    "sale_id": 3,
    "product_id": 1,
    "customer_id": 3,
    "quantity_sold": 5,
    "date": "2026-04-23"
  },
  {
    "sale_id": 4,
    "product_id": 3,
    "customer_id": 1,
    "quantity_sold": 10,
    "date": "2026-04-23"
  }
]
```